

Laravel Avancé : SOLID et HATEOAS

Le Guide Ultime pour une Architecture API Robuste

Auteur : Manus AI Date de publication : Novembre 2025

Partie I : Les Fondations d' une API Robuste

Chapitre 1 : Introduction et Mise en Place

1.1. Philosophie du Livre : Apprendre en Faisant (L' approche "Toyus Ca")

Ce livre est un guide pratique, conçu non pas pour une lecture passive, mais pour une **implémentation active**. Notre philosophie repose sur le principe de l' apprentissage par la pratique, que nous appelons l' approche "**Toyus Ca**" (du français "tout y est, ça marche").

Chaque concept théorique, qu' il s' agisse des principes SOLID ou du modèle de maturité de Richardson, sera immédiatement suivi d' un exemple de code concret dans le contexte du framework Laravel. L' objectif est de vous fournir des instructions si claires et précises que vous pourrez les appliquer à la lettre, sans fatigue, pour construire une application d' une qualité architecturale exceptionnelle.

Nous ne laisserons aucun principe de côté. Si un concept est essentiel à une architecture propre, il sera intégré et expliqué. Vous apprendrez à utiliser les fonctionnalités avancées de Laravel — **Traits, Jobs, Events, Listeners, Mutators, etc.**

— non pas comme des options, mais comme des outils fondamentaux pour respecter les principes de conception.

1.2. Pourquoi Laravel ?

Laravel est le framework PHP le plus populaire et le plus apprécié pour une raison simple : il rend le développement web à la fois puissant et agréable. Cependant, sa flexibilité peut parfois conduire à des architectures désordonnées. Ce livre vous montrera comment exploiter la puissance de Laravel tout en le guidant vers une architecture propre et maintenable.

Laravel est particulièrement adapté à notre objectif car il fournit nativement des outils qui facilitent l'application des principes SOLID :

Principe SOLID	Fonctionnalité Laravel Correspondante
SRP (Responsabilité Unique)	Actions/Services, Jobs, Notifications
OCP (Ouvert/Fermé)	Interfaces, Service Providers, Events/Listeners
LSP (Substitution de Liskov)	Interfaces, Contrats
ISP (Ségrégation d' Interfaces)	Interfaces fines, Traits
DIP (Inversion de Dépendances)	Service Container, Injection de Dépendances

1.3. Installation et Configuration de l' Environnement

Pour commencer, assurez-vous d' avoir PHP (version 8.2+ recommandée), Composer et un système de gestion de base de données (MySQL ou PostgreSQL) installés.

Étape 1 : Création du Projet

Ouvrez votre terminal et exécutez la commande suivante pour créer notre projet, que nous nommerons `mobile-money-api` :

```
composer create-project laravel/laravel mobile-money-api
cd mobile-money-api
```

Étape 2 : Configuration de la Base de Données

Ouvrez le fichier `.env` et configurez vos identifiants de base de données.

```
# Fichier : .env
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=mobile_money_db
DB_USERNAME=root
DB_PASSWORD=
```

Étape 3 : Installation de Laravel Passport pour l' Authentification JWT

Notre API utilise l' authentification JWT, comme spécifié dans la documentation fournie. Laravel Passport est l' outil idéal pour cela.

```
composer require laravel/passport
php artisan migrate
php artisan passport:install --api
```

Nous allons modifier le modèle `User` pour utiliser le trait `HasApiTokens` de Passport.

Fichier touché : `app/Models/User.php`

```
<?php
// app/Models/User.php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;
use Laravel\Passport\HasApiTokens; // <-- Ajouté

class User extends Authenticatable
{
    use HasApiTokens, HasFactory, Notifiable; // <-- Ajouté HasApiTokens

    // ... (le reste du modèle)
}
```

1.4. Le Projet “API Mobile Money” : Modèle de Données Simplifié

Le cœur de notre application est la gestion des utilisateurs, des comptes et des transactions. Nous allons créer les migrations et les modèles correspondants.

Étape 1 : Création des Modèles et Migrations

```
php artisan make:model Account -m
php artisan make:model Transaction -m
```

Étape 2 : Définition des Schémas de Base de Données

Nous allons modifier les migrations pour refléter la spécification de l’ API.

Fichier **touché** :

```
database/migrations/YYYY_MM_DD_HHMMSS_create_users_table.php
```

Nous allons ajouter les champs `telephone` , `nom` , `prenom` et `type` au modèle `User` .

```
// database/migrations/YYYY_MM_DD_HHMMSS_create_users_table.php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('users', function (Blueprint $table) {
            $table->id();
            $table->string('nom');
            $table->string('prenom');
            $table->string('telephone')->unique(); // Clé unique pour
l'authentification
            $table->string('email')->unique()->nullable();
            $table->enum('type', ['client', 'admin', 'marchand'])->
default('client');
            $table->string('password');
            $table->rememberToken();
            $table->timestamps();
        });
    }
    // ...
};
```

Fichier

touché

:

database/migrations/YYYY_MM_DD_HHMMSS_create_accounts_table.php

```
// database/migrations/YYYY_MM_DD_HHMMSS_create_accounts_table.php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('accounts', function (Blueprint $table) {
            $table->id();
            $table->foreignId('user_id')->constrained()-
>onDelete('cascade');
            $table->string('numero_compte')->unique();
            $table->decimal('solde', 15, 2)->default(0.00);
            $table->enum('type', ['courant', 'epargne', 'marchand'])-
>default('courant');
            $table->string('code_marchand')->unique()->nullable(); // Pour
les comptes marchands
            $table->timestamps();
        });
    }
    // ...
};
```

Fichier

touché

:

database/migrations/YYYY_MM_DD_HHMMSS_create_transactions_table.php

```
// database/migrations/YYYY_MM_DD_HHMMSS_create_transactions_table.php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('transactions', function (Blueprint $table) {
            $table->id();
            $table->foreignId('account_id')->constrained()-
>onDelete('cascade');
            $table->decimal('montant', 15, 2);
            $table->enum('type', ['depot', 'retrait', 'transfert']);
            $table->enum('statut', ['en_attente', 'succes', 'echec'])-
>default('succes');
            $table->text('description')->nullable();
            $table->timestamps();
        });
    }
    // ...
};
```

Étape 3 : Exécution des Migrations

```
php artisan migrate
```

Étape 4 : Mise à jour des Modèles

Nous allons ajouter les relations et les champs *fillable* aux modèles.

Fichier touché : `app/Models/User.php`

```
// app/Models/User.php (Mise à jour)

// ...
class User extends Authenticatable
{
    use HasApiTokens, HasFactory, Notifiable;

    protected $fillable = [
        'nom',
        'prenom',
        'telephone',
        'email',
        'type',
        'password',
    ];

    // Relation : Un utilisateur a plusieurs comptes
    public function accounts()
    {
        return $this->hasMany(Account::class);
    }
}
```

Fichier touché : app/Models/Account.php


```
// app/Models/Account.php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Account extends Model
{
    use HasFactory;

    protected $fillable = [
        'user_id',
        'numero_compte',
        'solde',
        'type',
        'code_marchand',
    ];

    // Relation : Un compte appartient à un utilisateur
    public function user()
    {
        return $this->belongsTo(User::class);
    }

    // Relation : Un compte a plusieurs transactions
    public function transactions()
    {
        return $this->hasMany(Transaction::class);
    }
}
```

Fichier touché : app/Models/Transaction.php

```
// app/Models/Transaction.php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Transaction extends Model
{
    use HasFactory;

    protected $fillable = [
        'account_id',
        'montant',
        'type',
        'statut',
        'description',
    ];

    // Relation : Une transaction appartient à un compte
    public function account()
    {
        return $this->belongsTo(Account::class);
    }
}
```

Chapitre 2 : Les Principes de Conception Incontournables

2.1. S.O.L.I.D. à la Lettre

Les principes **SOLID** sont le socle de toute architecture logicielle de qualité. Ils ont été popularisés par Robert C. Martin (Uncle Bob) et visent à rendre le code plus compréhensible, flexible et maintenable.

2.1.1. Single Responsibility Principle (SRP) : Contrôleurs Mince, Services Épais

Le **Principe de Responsabilité Unique** stipule qu' une classe ne devrait avoir qu' une seule raison d' être modifiée. En pratique, cela signifie que nos contrôleurs

(Controllers) ne doivent s'occuper que de la gestion de la requête HTTP (validation, appel du service, réponse) et que la logique métier doit être déléguée à des classes dédiées : les **Services** ou les **Actions**.

Mauvaise Pratique (Violation du SRP) :

Un contrôleur qui gère la requête, la validation, la logique de création de l'utilisateur, l'envoi d'un email, et la réponse.

Bonne Pratique (Respect du SRP) :

Nous allons créer un répertoire `app/Services` pour notre logique métier.

Fichier touché : `app/Services/UserService.php` (À créer)

```
// app/Services/UserService.php

namespace App\Services;

use App\Models\User;
use Illuminate\Support\Facades\Hash;

/**
 * Responsabilité : Gérer la logique métier relative aux utilisateurs.
 */
class UserService
{
    /**
     * Crée un nouvel utilisateur.
     * @param array $data
     * @return User
     */
    public function createUser(array $data): User
    {
        // 1. Logique de hachage du mot de passe (Responsabilité du Service)
        $data['password'] = Hash::make($data['password']);

        // 2. Création de l'utilisateur
        $user = User::create($data);

        // Note : L'envoi d'email ou la création de compte initial sera
        // délégué
        // à un Event/Listener ou un Job pour maintenir le SRP du
        // UserService.

        return $user;
    }

    // ... autres méthodes de logique métier (update, delete, etc.)
}

```

Le contrôleur devient alors très mince.

Fichier touché : `app/Http/Controllers/Api/UserController.php` (À créer)

```
php artisan make:controller Api/UserController --api
```

```
// app/Http/Controllers/Api/UserController.php

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Http\Requests\UserStoreRequest; // À créer
use App\Services\UserService;
use Illuminate\Http\JsonResponse;

class UserController extends Controller
{
    protected $userService;

    // Injection de Dépendances (DIP)
    public function __construct(UserService $userService)
    {
        $this->userService = $userService;
    }

    /**
     * 1. Gère la requête HTTP.
     * 2. Délègue la logique métier au Service.
     * 3. Retourne la réponse HTTP.
     */
    public function store(UserStoreRequest $request): JsonResponse
    {
        // Le contrôleur ne fait qu'appeler le service
        $user = $this->userService->createUser($request->validated());

        return response()->json([
            'status' => 'success',
            'message' => 'Utilisateur créé',
            'data' => $user,
        ], 201);
    }

    // ... autres méthodes (index, show, update, destroy)
}

```

2.1.2. Open/Closed Principle (OCP) : Utilisation des Interfaces

Le **Principe Ouvert/Fermé** stipule qu’ une entité logicielle (classe, module, fonction, etc.) doit être **ouverte à l’ extension, mais fermée à la modification**. Cela signifie

que nous devrions pouvoir ajouter de nouvelles fonctionnalités sans modifier le code existant qui fonctionne déjà.

Nous allons appliquer l' OCP en utilisant des **Interfaces** pour nos Repositories. Si nous décidons de changer notre base de données de MySQL à MongoDB, nous n' aurons qu' à créer une nouvelle implémentation de l' interface sans toucher au code qui utilise le Repository (comme notre `UserService`).

Fichier touché : `app/Contracts/UserRepositoryInterface.php` (À créer)

```
// app/Contracts/UserRepositoryInterface.php

namespace App\Contracts;

use App\Models\User;
use Illuminate\Database\Eloquent\Collection;

/**
 * Contrat pour la gestion de la persistance des utilisateurs.
 */
interface UserRepositoryInterface
{
    public function all(): Collection;
    public function find(int $id): ?User;
    public function create(array $data): User;
    public function update(int $id, array $data): bool;
    public function delete(int $id): bool;
}
```

Fichier touché : `app/Repositories/Eloquent/UserRepository.php` (À créer)

```
// app/Repositories/Eloquent/UserRepository.php

namespace App\Repositories\Eloquent;

use App\Contracts\UserRepositoryInterface;
use App\Models\User;
use Illuminate\Database\Eloquent\Collection;

/**
 * Implémentation du contrat pour Eloquent (MySQL/PostgreSQL).
 */
class UserRepository implements UserRepositoryInterface
{
    public function all(): Collection
    {
        return User::all();
    }

    public function find(int $id): ?User
    {
        return User::find($id);
    }

    public function create(array $data): User
    {
        return User::create($data);
    }

    public function update(int $id, array $data): bool
    {
        return $this->find($id)?->update($data) ?? false;
    }

    public function delete(int $id): bool
    {
        return $this->find($id)?->delete() ?? false;
    }
}
```

Nous allons ensuite lier l' interface à son implémentation dans un **Service Provider** (DIP).

Fichier touché : app/Providers/RepositoryServiceProvider.php (À créer)

```
php artisan make:provider RepositoryServiceProvider
```

```
// app/Providers/RepositoryServiceProvider.php

namespace App\Providers;

use App\Contracts\UserRepositoryInterface;
use App\Repositories\Eloquent\UserRepository;
use Illuminate\Support\ServiceProvider;

class RepositoryServiceProvider extends ServiceProvider
{
    public function register(): void
    {
        // Liaison de l'Interface à l'Implémentation
        $this->app->bind(
            UserRepositoryInterface::class,
            UserRepository::class
        );
    }

    // ...
}
```

N'oubliez pas d'enregistrer ce nouveau Service Provider dans `config/app.php`.

Fichier touché : `config/app.php`

```
// config/app.php

// ...
'providers' => ServiceProvider::defaultProviders()->merge([
    // ...
    App\Providers\RepositoryServiceProvider::class, // <-- Ajouté
])->toArray(),
// ...
```

Maintenant, si nous injectons `UserRepositoryInterface` dans notre `UserService`, nous respectons l'OCP : le `UserService` est fermé à la modification (il ne se soucie

pas de la base de données utilisée), mais le système est ouvert à l'extension (nous pouvons créer un `MongoUserRepository` et le lier dans le Service Provider).

Fichier touché : `app/Services/UserService.php` (Mise à jour)

```
// app/Services/UserService.php (Mise à jour)

namespace App\Services;

use App\Contracts\UserRepositoryInterface; // <-- Importé
use App\Models\User;
use Illuminate\Support\Facades\Hash;

class UserService
{
    protected $userRepository;

    // Injection de l'Interface (DIP)
    public function __construct(UserRepositoryInterface $userRepository)
    {
        $this->userRepository = $userRepository;
    }

    public function createUser(array $data): User
    {
        $data['password'] = Hash::make($data['password']);

        // Utilisation du Repository via l'Interface
        $user = $this->userRepository->create($data);

        return $user;
    }

    // ...
}
```

2.1.3. Liskov Substitution Principle (LSP) : Les Contrats

Le **Principe de Substitution de Liskov** (LSP) garantit que les objets d'un programme peuvent être remplacés par des instances de leurs sous-types sans altérer la justesse du programme. En PHP, cela se traduit par le respect des signatures de méthodes définies dans les interfaces ou les classes mères.

Dans notre exemple précédent, l' interface `UserRepositoryInterface` agit comme un contrat. Toute classe qui implémente cette interface (comme `UserRepository`) doit respecter les signatures de méthodes (noms, types de paramètres, types de retour). Si `UserRepository` ne retournerait pas un `User` dans sa méthode `find()`, il violerait le LSP, car le code qui s' attend à un `User` (comme notre `UserService`) pourrait planter.

Application dans Laravel :

Laravel utilise le LSP partout, notamment avec les **Contrats** (Contracts). Par exemple, le contrat `Illuminate\Contracts\Queue\ShouldQueue` garantit que toute classe qui l' implémente peut être mise en file d' attente par le système de queue de Laravel.

2.1.4. Interface Segregation Principle (ISP) : Interfaces Spécifiques

Le **Principe de Ségrégation d' Interfaces** stipule qu' un client ne devrait pas être forcé de dépendre d' interfaces qu' il n' utilise pas. Il est préférable d' avoir plusieurs interfaces spécifiques (fines) plutôt qu' une seule interface générale (grosse).

Mauvaise Pratique (Violation de l' ISP) :

Une seule interface `CrudRepositoryInterface` avec toutes les méthodes (créer, lire, mettre à jour, supprimer, lister, filtrer, paginer, etc.). Si un service n' a besoin que de la lecture, il dépendra inutilement des méthodes de modification.

Bonne Pratique (Respect de l' ISP) :

Nous pourrions créer des interfaces plus fines :

- `ReadableRepositoryInterface` (pour `find`, `all`)
- `WritableRepositoryInterface` (pour `create`, `update`, `delete`)
- `PaginableRepositoryInterface` (pour `paginate`, `filter`)

Un service qui n' a besoin que de lire les données n' injectera que `ReadableRepositoryInterface`.

2.1.5. Dependency Inversion Principle (DIP) : Abstractions vs Implémentations

Le **Principe d' Inversion de Dépendances** (DIP) est le plus crucial pour une architecture propre. Il a deux règles :

1. Les modules de haut niveau ne doivent pas dépendre des modules de bas niveau. Les deux doivent dépendre d' abstractions.
2. Les abstractions ne doivent pas dépendre des détails. Les détails doivent dépendre des abstractions.

En d' autres termes, le code de haut niveau (la logique métier dans `UserService`) ne doit pas connaître les détails d' implémentation (comment les données sont stockées dans `UserRepository`). Il doit dépendre d' une abstraction (`UserRepositoryInterface`).

Nous avons déjà appliqué le DIP en utilisant le **Service Container** de Laravel pour lier l' interface (`UserRepositoryInterface`) à son implémentation (`UserRepository`) dans le `RepositoryServiceProvider` .

Tableau Récapitulatif de l' Architecture SOLID :

Couche	Rôle	Principe SOLID Appliqué	Fichiers Clés
Contrôleur	Gestion de la requête HTTP et de la réponse.	SRP (Contrôleur Mince)	UserController.php
Service	Logique métier pure.	SRP, DIP (Dépend de l' Interface)	UserService.php
Contrat	Définition des méthodes (Abstraction).	OCP, LSP, ISP, DIP	UserRepositoryInterface.php
Repository	Détails de la persistance (Implémentation).	OCP, DIP (Dépend de l' Abstraction)	UserRepository.php
Service Provider	Liaison de l' Abstraction à l' Implémentation.	DIP	RepositoryServiceProvider.php

2.2. Le Modèle de Maturité de Richardson (RMM)

Le Modèle de Maturité de Richardson (RMM) est un modèle en quatre étapes qui définit les niveaux de maturité d' une API REST. Notre objectif est d' atteindre le **Niveau 4**, souvent appelé le “Saint Graal” du REST, en raison de l' utilisation de **HATEOAS**.

2.2.1. Les Niveaux 0, 1, 2, 3 : Rappel Rapide

Niveau	Caractéristique	Description
Niveau 0	Le marais du Protocole	Utilisation d' un seul URI et d' une seule méthode HTTP (souvent POST) pour toutes les opérations (ex: SOAP).
Niveau 1	Ressources	Introduction des ressources (URI multiples). Chaque ressource a son propre URI.
Niveau 2	Verbes HTTP	Utilisation des verbes HTTP (GET, POST, PUT, DELETE) pour les actions CRUD. Utilisation des codes de statut HTTP.
Niveau 3	Hypermedia Controls (HATEOAS)	Ajout de liens hypermédia dans la réponse pour guider le client vers les actions possibles. C' est le niveau que nous visons.

2.2.2. Atteindre le Niveau 4 (HATEOAS) : Le Saint Graal du REST

Le niveau 3 du RMM est souvent appelé **HATEOAS** (Hypermedia As The Engine Of Application State).

***HATEOAS** est un contrainte de l' architecture REST qui permet au client de l' API de découvrir dynamiquement les actions possibles en se basant sur les liens inclus dans la réponse du serveur, sans avoir besoin de connaître l' ensemble des URI à l' avance.*

Dans notre spécification “API Mobile Money” , nous avons déjà vu des exemples de HATEOAS :

```
// Extrait de la réponse de connexion
"user": {
  // ...
  "links": {
    "self": "/api/utilisateurs/1",
    "comptes": "/api/utilisateurs/1/comptes",
    "transactions": "/api/utilisateurs/1/transactions"
  }
}
```

Ces liens (`self`, `comptes`, `transactions`) sont les **Hypermedia Controls**. Ils indiquent au client où aller ensuite.

Implémentation avec Laravel Resources :

Laravel facilite l'implémentation de HATEOAS grâce aux **API Resources**. Nous allons créer une ressource pour notre modèle `User`.

Fichier touché : `app/Http/Resources/UserResource.php` (À créer)

```
php artisan make:resource UserResource
```

```
// app/Http/Resources/UserResource.php

namespace App\Http\Resources;

use Illuminate\Http\Request;
use Illuminate\Http\Resources\Json\JsonResource;

class UserResource extends JsonResource
{
    /**
     * Transforme la ressource en un tableau.
     * @param Request $request
     * @return array<string, mixed>
     */
    public function toArray(Request $request): array
    {
        return [
            'id' => $this->id,
            'nom' => $this->nom,
            'prenom' => $this->prenom,
            'telephone' => $this->telephone,
            'email' => $this->email,
            'type' => $this->type,
            // Les liens HATEOAS
            'links' => $this->links(),
        ];
    }

    /**
     * Génère les liens hypermédia pour la ressource.
     * @return array
     */
    protected function links(): array
    {
        // Nous utilisons la fonction route() pour générer des URI absolus
        // et robustes
        return [
            'self' => route('users.show', $this->id),
            'comptes' => route('users.accounts.index', $this->id),
            // Ajout conditionnel : l'admin peut supprimer
            'delete' => $this->when(auth()->user()->type === 'admin',
function () {
                return route('users.destroy', $this->id);
            }
        );
    }
}
```

```
}  
}
```

Mise à jour des Routes :

Pour que `route('users.show', ...)` fonctionne, nous devons nommer nos routes.

Fichier touché : `routes/api.php`

```
// routes/api.php  
  
use App\Http\Controllers\Api\UserController;  
  
Route::middleware('auth:api')->group(function () {  
    // Routes Utilisateurs (Nommées pour HATEOAS)  
    Route::apiResource('utilisateurs', UserController::class)->names([  
        'index' => 'users.index',  
        'store' => 'users.store',  
        'show' => 'users.show',  
        'update' => 'users.update',  
        'destroy' => 'users.destroy',  
    ]);  
  
    // Route pour lister les comptes d'un utilisateur (nécessaire pour le  
    lien 'comptes')  
    Route::get('utilisateurs/{user}/comptes', [AccountController::class,  
        'indexByUser'])  
        ->name('users.accounts.index'); // À créer  
});
```

En utilisant `UserResource` dans notre contrôleur, nous garantissons que chaque réponse d'utilisateur inclut les liens nécessaires, atteignant ainsi le **Niveau 4 (HATEOAS)** du Modèle de Maturité de Richardson.

Partie II : Architecture et Implémentation

Chapitre 3 : Structurer le Code avec SOLID

3.1. Les Couches de l' Application : Contrôleurs, Services, Repositories

Pour maintenir le SRP et le DIP, nous allons formaliser notre architecture en trois couches principales :

1. **Couche Présentation (Controllers)** : Gère les requêtes et les réponses HTTP.
2. **Couche Métier (Services/Actions)** : Contient la logique métier pure.
3. **Couche Persistance (Repositories)** : Gère l' accès aux données.

Cette séparation garantit que la logique métier est indépendante du framework (Laravel) et de la base de données (Eloquent).

3.2. Mise en place des Services (SRP) : La Logique Métier

Les Services sont le cœur de notre application. Ils orchestrent les opérations en utilisant les Repositories.

Nous allons créer un service pour la gestion des comptes.

Fichier touché : `app/Contracts/AccountRepositoryInterface.php` (À créer)

```
// app/Contracts/AccountRepositoryInterface.php

namespace App\Contracts;

use App\Models\Account;
use Illuminate\Database\Eloquent\Collection;

interface AccountRepositoryInterface
{
    public function find(int $id): ?Account;
    public function create(array $data): Account;
    public function findByUserId(int $userId): Collection;
    // ...
}
```

Fichier touché : app/Repositories/Eloquent/AccountRepository.php (À créer)

```
// app/Repositories/Eloquent/AccountRepository.php

namespace App\Repositories\Eloquent;

use App\Contracts\AccountRepositoryInterface;
use App\Models\Account;
use Illuminate\Database\Eloquent\Collection;

class AccountRepository implements AccountRepositoryInterface
{
    public function find(int $id): ?Account
    {
        return Account::find($id);
    }

    public function create(array $data): Account
    {
        // Génération d'un numéro de compte unique avant la création
        $data['numero_compte'] = 'CPT-' . str_pad(rand(1, 99999), 5, '0',
STR_PAD_LEFT);
        return Account::create($data);
    }

    public function findById(int $userId): Collection
    {
        return Account::where('user_id', $userId)->get();
    }
}
```

Fichier touché : app/Services/AccountService.php (À créer)

```
// app/Services/AccountService.php

namespace App\Services;

use App\Contracts\AccountRepositoryInterface;
use App\Models\Account;
use Illuminate\Validation\ValidationException;

class AccountService
{
    protected $accountRepository;

    public function __construct(AccountRepositoryInterface
$accountRepository)
    {
        $this->accountRepository = $accountRepository;
    }

    /**
     * Crée un compte en appliquant les règles métier.
     * @param array $data
     * @return Account
     * @throws ValidationException
     */
    public function createAccount(array $data): Account
    {
        // Règle métier : Un client ne peut avoir qu'un compte actif par
type
        if ($data['type'] === 'courant' && $this-
>hasActiveAccountOfType($data['user_id'], 'courant')) {
            throw ValidationException::withMessages([
                'type' => 'L'utilisateur a déjà un compte courant actif.'
            ]);
        }

        // Règle métier : Commerçants : code_marchand obligatoire et unique
        if ($data['type'] === 'marchand' && empty($data['code_marchand'])) {
            throw ValidationException::withMessages([
                'code_marchand' => 'Le code marchand est obligatoire pour un
compte marchand.'
            ]);
        }

        return $this->accountRepository->create($data);
    }
}
```

```
/**
 * Vérifie si l'utilisateur a déjà un compte du type spécifié.
 * @param int $userId
 * @param string $type
 * @return bool
 */
protected function hasActiveAccountOfType(int $userId, string $type):
bool
{
    // Logique de vérification dans le service, utilisant le repository
    si nécessaire
    $accounts = $this->accountRepository->findByUserId($userId);
    return $accounts->where('type', $type)->isNotEmpty();
}
```

Mise à jour du Service Provider :

Fichier touché : app/Providers/RepositoryServiceProvider.php (Mise à jour)

```
// app/Providers/RepositoryServiceProvider.php (Mise à jour)

// ...
use App\Contracts\AccountRepositoryInterface; // <-- Ajouté
use App\Repositories\Eloquent\AccountRepository; // <-- Ajouté

class RepositoryServiceProvider extends ServiceProvider
{
    public function register(): void
    {
        $this->app->bind(
            UserRepositoryInterface::class,
            UserRepository::class
        );

        // Liaison pour le AccountRepository
        $this->app->bind(
            AccountRepositoryInterface::class,
            AccountRepository::class
        );
    }

    // ...
}
```

3.3. Mise en place des Actions/Jobs pour les Tâches Complexes (SRP)

Pour les opérations complexes qui impliquent plusieurs étapes et qui doivent être transactionnelles, nous allons utiliser des **Actions** (ou Jobs synchrones) pour encapsuler la logique. Cela permet de maintenir le SRP du `UserService` et du `AccountService`.

Considérons l'opération de **Transfert d'Argent**, qui est l'opération métier la plus critique. Elle implique :

1. Vérification des soldes.
2. Débit du compte source.
3. Crédit du compte destination.
4. Enregistrement de deux transactions.
5. Notification des utilisateurs.

Toute cette logique doit être dans une seule unité de travail (SRP) et doit être transactionnelle.

Fichier touché : `app/Actions/ProcessTransferAction.php` (À créer)

```
php artisan make:class Actions/ProcessTransferAction
```

```
// app/Actions/ProcessTransferAction.php

namespace App\Actions;

use App\Models\Account;
use Illuminate\Support\Facades\DB;
use Illuminate\Validation\ValidationException;

/**
 * Responsabilité : Exécuter un transfert d'argent de manière
 transactionnelle.
 */
class ProcessTransferAction
{
    public function execute(Account $source, Account $destination, float
$amount): array
    {
        // 1. Vérification du solde
        if ($source->solde < $amount) {
            throw ValidationException::withMessages([
                'solde' => 'Solde insuffisant pour effectuer le transfert.'
            ]);
        }

        // 2. Opération transactionnelle (garantit l'atomicité)
        DB::beginTransaction();
        try {
            // Débit du compte source
            $source->decrement('solde', $amount);
            $source->transactions()->create([
                'montant' => -$amount,
                'type' => 'transfert',
                'description' => 'Transfert sortant vers ' . $destination->numero_compte,
            ]);

            // Crédit du compte destination
            $destination->increment('solde', $amount);
            $destination->transactions()->create([
                'montant' => $amount,
                'type' => 'transfert',
                'description' => 'Transfert entrant de ' . $source->numero_compte,
            ]);
        }
    }
}
```



```

        DB::commit();

        // 3. Déclenchement d'un événement pour la notification (SRP)
        // event(new TransferProcessed($source, $destination, $amount));
        // À créer

        return [
            'status' => 'success',
            'message' => 'Transfert effectué avec succès.'
        ];

    } catch (\Exception $e) {
        DB::rollBack();
        // Log de l'erreur
        // throw $e;
        return [
            'status' => 'error',
            'message' => 'Échec du transfert : ' . $e->getMessage()
        ];
    }
}
}

```

Le contrôleur appellera simplement cette Action, maintenant le SRP.

3.4. Les Traits : Réutilisation de Code (OCP/ISP)

Les **Traits** de PHP permettent de réutiliser des ensembles de méthodes dans différentes classes, sans utiliser l'héritage. Ils sont parfaits pour appliquer l'OCP et l'ISP en ajoutant des fonctionnalités spécifiques à un modèle ou une classe sans la surcharger.

Nous allons créer un Trait pour la gestion de l'Audit Log, car l'API spécifie que de nombreuses opérations doivent être auditées.

Fichier touché : app/Traits/Auditable.php (À créer)

```
// app/Traits/Auditable.php

namespace App\Traits;

use App\Models\AuditLog; // À créer

trait Auditable
{
    /**
     * Enregistre une entrée d'audit pour l'opération.
     * @param string $action
     * @param array $details
     */
    public function logAudit(string $action, array $details = []): void
    {
        // Nous utiliserons un Job pour l'enregistrement réel (SRP)
        // dispatch(new LogAuditJob($this, $action, $details)); // À créer

        // Pour l'instant, une implémentation simple
        \Log::info("AUDIT: " . class_basename($this) . " ID: " . $this->id .
            " - Action: " . $action, $details);
    }
}
```

Nous pouvons maintenant utiliser ce Trait dans nos modèles `User` et `Account` .

Fichier touché : `app/Models/User.php` (Mise à jour)

```
// app/Models/User.php (Mise à jour)

// ...
use App\Traits\Auditable; // <-- Ajouté

class User extends Authenticatable
{
    use HasApiTokens, HasFactory, Notifiable, Auditable; // <-- Ajouté
    Auditable

    // ...
}
```

Chaque fois qu' une opération est effectuée sur un utilisateur, nous pouvons appeler `$user->logAudit('update_profile', $changes);` dans notre `UserService`, sans que le modèle `User` ne soit responsable de la logique d' audit elle-même (SRP).

Chapitre 4 : Le Cycle de Vie d' une Requête Laravel

4.1. Les Requests : Validation et Préparation des Données

Les **Form Requests** de Laravel sont l' endroit idéal pour la validation des données d' entrée. Elles respectent le SRP en séparant la logique de validation du contrôleur.

Nous allons créer la Request pour la création d' un utilisateur.

Fichier touché : `app/Http/Requests/UserStoreRequest.php` (À créer)

```
php artisan make:request UserStoreRequest
```

```
// app/Http/Requests/UserStoreRequest.php

namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;

class UserStoreRequest extends FormRequest
{
    /**
     * Détermine si l'utilisateur est autorisé à faire cette requête.
     */
    public function authorize(): bool
    {
        // Règle métier : Seul un administrateur peut créer un utilisateur
        // via cette route
        return auth()->check() && auth()->user()->type === 'admin';
    }

    /**
     * Obtient les règles de validation qui s'appliquent à la requête.
     */
    public function rules(): array
    {
        return [
            'nom' => ['required', 'string', 'max:255'],
            'prenom' => ['required', 'string', 'max:255'],
            'telephone' => ['required', 'string', 'unique:users,telephone'],
            // Unicité
            'email' => ['nullable', 'email', 'unique:users,email'],
            'password' => ['required', 'string', 'min:8'],
            'type' => ['required', Rule::in(['client', 'admin',
            'marchand'])),
        ];
    }
}
```

Le contrôleur utilise simplement `$request->validated()`, ce qui garantit que seules les données validées et autorisées atteignent le `UserService`.

4.2. Les Middlewares : Sécurité et Pré-traitement

Les **Middlewares** sont des filtres qui inspectent et traitent les requêtes HTTP entrantes. Ils sont essentiels pour l'authentification et l'autorisation.

Notre spécification exige que certaines routes soient réservées à l'administrateur. Nous allons créer un Middleware pour cela.

Fichier touché : `app/Http/Middleware/AdminOnly.php` (À créer)

```
php artisan make:middleware AdminOnly
```

```
// app/Http/Middleware/AdminOnly.php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class AdminOnly
{
    /**
     * Gère une requête entrante.
     */
    public function handle(Request $request, Closure $next): Response
    {
        if (!auth()->check() || auth()->user()->type !== 'admin') {
            return response()->json([
                'status' => 'error',
                'message' => 'Accès refusé. Seuls les administrateurs sont
autorisés.'
            ], 403);
        }

        return $next($request);
    }
}
```

Enregistrement du Middleware :

Fichier touché : app/Http/Kernel.php

```
// app/Http/Kernel.php

// ...
protected $middlewareAliases = [
    // ...
    'admin' => \App\Http\Middleware\AdminOnly::class, // <-- Ajouté
];
// ...
```

Nous pouvons maintenant l'appliquer à nos routes :

Fichier touché : routes/api.php (Mise à jour)

```
// routes/api.php (Mise à jour)

// ...
Route::middleware(['auth:api', 'admin'])->group(function () {
    // Seul l'admin peut lister et créer des utilisateurs
    Route::get('utilisateurs', [UserController::class, 'index'])->
>name('users.index');
    Route::post('utilisateurs', [UserController::class, 'store'])->
>name('users.store');
});

Route::middleware('auth:api')->group(function () {
    // Les autres opérations CRUD peuvent être gérées par l'admin OU le
    propriétaire
    Route::get('utilisateurs/{user}', [UserController::class, 'show'])->
>name('users.show');
    // ...
});
```

4.3. Les Ressources API : Transformer les Modèles pour la Réponse

Les **API Resources** (vues dans la section HATEOAS) sont cruciales pour le SRP dans la couche de présentation. Elles garantissent que la structure de la réponse JSON est séparée du modèle Eloquent.

Nous allons créer la ressource pour le compte.

Fichier touché : `app/Http/Resources/AccountResource.php` (À créer)

```
php artisan make:resource AccountResource
```

```
// app/Http/Resources/AccountResource.php

namespace App\Http\Resources;

use Illuminate\Http\Request;
use Illuminate\Http\Resources\Json\JsonResource;

class AccountResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        return [
            'id' => $this->id,
            'numeroCompte' => $this->numero_compte,
            'solde' => $this->solde,
            'type' => $this->type,
            'utilisateur_id' => $this->user_id,
            // Inclusion conditionnelle du code marchand
            'code_marchand' => $this->when($this->type === 'marchand',
$this->code_marchand),
            // Relation utilisateur (chargement paresseux si non chargé)
            'utilisateur' => UserResource::make($this->whenLoaded('user')),
            'links' => $this->links(),
        ];
    }

    protected function links(): array
    {
        return [
            'self' => route('accounts.show', $this->id),
            'transactions' => route('accounts.transactions.index', $this->id),
            'utilisateur' => route('users.show', $this->user_id),
        ];
    }
}
```

4.4. Implémentation du HATEOAS (Niveau 4 RMM)

Pour atteindre le niveau 4, nous devons nous assurer que les liens sont présents dans toutes les réponses pertinentes.

Exemple de Collection de Ressources avec HATEOAS :

Lorsque nous listons les comptes, nous devons inclure les liens de pagination et les liens pour chaque ressource.

Fichier touché : `app/Http/Resources/AccountCollection.php` (À créer)

```
php artisan make:resource AccountCollection --collection
```



```
// app/Http/Resources/AccountCollection.php

namespace App\Http\Resources;

use Illuminate\Http\Request;
use Illuminate\Http\Resources\Json\ResourceCollection;

class AccountCollection extends ResourceCollection
{
    public function toArray(Request $request): array
    {
        return [
            'status' => 'success',
            'data' => $this->collection,
            // Les liens de pagination sont automatiquement gérés par
Laravel
            'meta' => [
                'total' => $this->total(),
                'count' => $this->count(),
                'per_page' => $this->perPage(),
                'current_page' => $this->currentPage(),
                'total_pages' => $this->lastPage(),
            ],
            'links' => [
                'self' => $this->url($this->currentPage()),
                'first' => $this->url(1),
                'last' => $this->url($this->lastPage()),
                'prev' => $this->previousPageUrl(),
                'next' => $this->nextPageUrl(),
            ],
        ];
    }
}
```

En utilisant `AccountCollection::make($accounts->paginate(10))` dans le contrôleur, nous fournissons une réponse JSON complète qui inclut les données, les métadonnées de pagination, et les liens hypermédia pour naviguer dans la collection, respectant ainsi le RMM Niveau 4.

Chapitre 5 : Les Fonctionnalités Avancées de Laravel

5.1. Les Mutators et Accessors : Manipulation des Attributs du Modèle

Les **Mutators** et **Accessors** de Laravel permettent de modifier la valeur d'un attribut lors de sa récupération (Accessor) ou de son enregistrement (Mutator). Ils sont parfaits pour encapsuler la logique de formatage ou de transformation au niveau du modèle (SRP).

Exemple : Masquer le numéro de téléphone pour les non-administrateurs

Nous allons créer un Accessor pour le champ `telephone` dans le modèle `User`.

Fichier touché : `app/Models/User.php` (Mise à jour)

```
// app/Models/User.php (Mise à jour)

// ...
use Illuminate\Database\Eloquent\Casts\Attribute; // Pour les nouveaux
Accessors/Mutators

class User extends Authenticatable
{
    // ...

    /**
     * Accesseur pour le numéro de téléphone.
     * Masque le numéro si l'utilisateur connecté n'est pas un admin.
     */
    protected function telephone(): Attribute
    {
        return Attribute::make(
            get: fn (string $value) => (auth()->check() && auth()->user()-
>type === 'admin')
                ? $value
                : substr($value, 0, 2) . '*****' . substr($value, -2),
        );
    }
}
```

Exemple : Hacher le mot de passe (Mutator)

Bien que nous l' ayons fait dans le `UserService` , le Mutator est la méthode la plus propre pour garantir que le mot de passe est toujours haché avant d' être enregistré.

Fichier touché : `app/Models/User.php` (Mise à jour)

```
// app/Models/User.php (Mise à jour)

// ...
use Illuminate\Support\Facades\Hash;

class User extends Authenticatable
{
    // ...

    /**
     * Mutateur pour le mot de passe.
     */
    protected function password(): Attribute
    {
        return Attribute::make(
            set: fn (string $value) => Hash::make($value),
        );
    }
}
```

En utilisant le Mutator, nous pouvons retirer la ligne de hachage du `UserService` , simplifiant ainsi le service et renforçant le SRP du modèle : le modèle est responsable de la manière dont ses propres données sont stockées.

5.2. Les Jobs et Queues : Traitement Asynchrone (SRP)

Les **Jobs** (ou Tâches) sont des classes qui encapsulent une tâche spécifique. Ils peuvent être exécutés immédiatement (synchrone) ou mis en file d' attente (asynchrone) pour un traitement ultérieur. Ils sont essentiels pour le SRP en déchargeant le cycle de requête HTTP des tâches longues.

Exemple : Envoi d' un email de bienvenue

L' envoi d' un email peut prendre du temps et ne devrait pas bloquer la réponse HTTP de la création d' utilisateur.

Fichier touché : app/Jobs/SendWelcomeEmail.php (À créer)

```
php artisan make:job SendWelcomeEmail
```

```
// app/Jobs/SendWelcomeEmail.php

namespace App\Jobs;

use App\Models\User;
use Illuminate\Bus\Queueable;
use Illuminate\Contracts\Queue\ShouldQueue;
use Illuminate\Foundation\Bus\Dispatchable;
use Illuminate\Queue\InteractsWithQueue;
use Illuminate\Queue\SerializesModels;
use Illuminate\Support\Facades\Mail; // À utiliser avec une Mailable

class SendWelcomeEmail implements ShouldQueue
{
    use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;

    protected $user;

    public function __construct(User $user)
    {
        $this->user = $user;
    }

    /**
     * Exécute le job.
     */
    public function handle(): void
    {
        // Logique d'envoi d'email (Mailable à créer)
        // Mail::to($this->user->email)->send(new WelcomeMail($this->user));
        \Log::info("Email de bienvenue envoyé à l'utilisateur ID: " . $this->user->id);
    }
}
```

Utilisation dans le Service :

Fichier touché : app/Services/UserService.php (Mise à jour)

```
// app/Services/UserService.php (Mise à jour)

// ...
use App\Jobs\SendWelcomeEmail; // <-- Importé

class UserService
{
    // ...

    public function createUser(array $data): User
    {
        // Le Mutator gère le hachage du mot de passe
        $user = $this->userRepository->create($data);

        // Déclenchement du Job asynchrone (ne bloque pas la requête)
        SendWelcomeEmail::dispatch($user);

        return $user;
    }

    // ...
}
```

Le `UserService` est maintenant plus propre : il crée l' utilisateur et délègue la tâche de notification à un Job, respectant le SRP.

Partie III : Gestion des Événements et des Transactions

Chapitre 6 : Événements et Écouteurs (Events & Listeners)

6.1. Le Principe de Découplage (OCP)

Les **Événements** et **Écouteurs** de Laravel sont l'outil parfait pour le découplage et l'application du **Principe Ouvert/Fermé (OCP)**. Lorsqu'un événement se produit (ex: `UserCreated`), le code qui déclenche l'événement (`UserService`) est fermé à la modification. Les nouvelles fonctionnalités (ex: envoyer un SMS, créer un compte par défaut) peuvent être ajoutées en créant de nouveaux Écouteurs, sans modifier le `UserService`.

6.2. Création d'un Événement

Nous allons créer un événement pour signaler qu'un utilisateur a été créé.

Fichier touché : `app/Events/UserCreated.php` (À créer)

```
php artisan make:event UserCreated
```

```
// app/Events/UserCreated.php

namespace App\Events;

use App\Models\User;
use Illuminate\Broadcasting\InteractsWithSockets;
use Illuminate\Foundation\Events\Dispatchable;
use Illuminate\Queue\SerializesModels;

class UserCreated
{
    use Dispatchable, InteractsWithSockets, SerializesModels;

    public $user;

    public function __construct(User $user)
    {
        $this->user = $user;
    }
}
```

6.3. Création d' un Écouteur

Nous allons créer un écouteur pour créer automatiquement un compte courant par défaut pour le nouvel utilisateur.

Fichier touché : `app/Listeners/CreateDefaultAccount.php` (À créer)

```
php artisan make:listener CreateDefaultAccount
```

```
// app/Listeners/CreateDefaultAccount.php

namespace App\Listeners;

use App\Events\UserCreated;
use App\Services\AccountService; // Utilisation du Service (DIP)
use Illuminate\Contracts\Queue\ShouldQueue;

class CreateDefaultAccount implements ShouldQueue // Exécuté en asynchrone
{
    protected $accountService;

    public function __construct(AccountService $accountService)
    {
        $this->accountService = $accountService;
    }

    /**
     * Gère l'événement.
     */
    public function handle(UserCreated $event): void
    {
        // Logique de création de compte par défaut
        $this->accountService->createAccount([
            'user_id' => $event->user->id,
            'type' => 'courant',
            'solde' => 0.00,
        ]);

        $event->user->logAudit('default_account_created');
    }
}
```

6.4. Enregistrement de l' Événement et de l' Écouteur

Nous devons lier l' événement à son écouteur dans le `EventServiceProvider`.

Fichier touché : `app/Providers/EventServiceProvider.php`


```
// app/Providers/EventServiceProvider.php

// ...
protected $listen = [
    \App\Events\UserCreated::class => [
        \App\Listeners\CreateDefaultAccount::class,
        // On pourrait ajouter d'autres écouteurs ici sans toucher au
UserService (OCP)
        // \App\Listeners\SendSmsNotification::class,
    ],
];
// ...
```

6.5. Déclenchement de l' Événement

Nous mettons à jour le `UserService` pour déclencher l' événement au lieu d' appeler directement le Job d' email.

Fichier touché : `app/Services/UserService.php` (Mise à jour)

```
// app/Services/UserService.php (Mise à jour)

// ...
use App\Events\UserCreated; // <-- Importé

class UserService
{
    // ...

    public function createUser(array $data): User
    {
        $user = $this->userRepository->create($data);

        // Déclenchement de l'événement (au lieu du Job d'email)
        UserCreated::dispatch($user);

        return $user;
    }

    // ...
}
```

Le `UserService` est maintenant encore plus découplé. Il ne sait pas ce qui se passe après la création de l'utilisateur ; il se contente de signaler que l'événement `UserCreated` a eu lieu. C'est l'essence de l'OCP.

Chapitre 7 : Transactions et Cohérence des Données

7.1. Gestion des Transactions de Base de Données

L'atomicité des opérations est vitale dans une application financière. Nous avons déjà vu l'utilisation de `DB::beginTransaction()` dans notre `ProcessTransferAction`.

Rappel :

```
// Extrait de ProcessTransferAction.php
DB::beginTransaction();
try {
    // Opérations de débit et de crédit
    DB::commit();
} catch (\Exception $e) {
    DB::rollBack();
    // Gestion de l'erreur
}
```

7.2. Les Jobs Chaînés (Chainable Jobs) pour les Opérations Multi-Étapes

Pour les opérations asynchrones qui doivent s'exécuter dans un ordre précis, les **Jobs Chaînés** (Job Chaining) sont la solution.

Exemple : Processus de Fin de Journée (End-of-Day)

1. Calculer les intérêts (Job 1).
2. Enregistrer les transactions d'intérêts (Job 2).
3. Envoyer un rapport aux administrateurs (Job 3).

Fichier touché : `app/Jobs/CalculateInterest.php` (À créer)

```
php artisan make:job CalculateInterest
```

Fichier touché : `app/Jobs/GenerateEndOfDayReport.php` (À créer)

```
php artisan make:job GenerateEndOfDayReport
```

Chaînage des Jobs :

```
// Dans un contrôleur ou un service de planification
use App\Jobs\CalculateInterest;
use App\Jobs\GenerateEndOfDayReport;

// Le job 2 (LogInterestTransaction) doit être créé
$jobs = [
    new CalculateInterest(),
    new LogInterestTransaction(), // Job à créer
    new GenerateEndOfDayReport(),
];

// Dispatcher la chaîne
\Illuminate\Support\Facades\Bus::chain($jobs)->dispatch();
```

Le chaînage garantit que si un Job échoue, les Jobs suivants ne seront pas exécutés, maintenant la cohérence du processus.

7.3. Les Notifications : Informer l' Utilisateur

Laravel simplifie l' envoi de notifications via différents canaux (email, SMS, base de données, etc.).

Exemple : Notification de Transfert Réussi

Fichier touché : `app/Notifications/TransferSuccess.php` (À créer)

```
php artisan make:notification TransferSuccess
```

```
// app/Notifications/TransferSuccess.php

namespace App\Notifications;

use App\Models\Transaction;
use Illuminate\Bus\Queueable;
use Illuminate\Notifications\Notification;
use Illuminate\Notifications\Messages\MailMessage;

class TransferSuccess extends Notification
{
    use Queueable;

    protected $transaction;

    public function __construct(Transaction $transaction)
    {
        $this->transaction = $transaction;
    }

    /**
     * Obtient les canaux de notification.
     */
    public function via(object $notifiable): array
    {
        return ['mail']; // Peut être ['mail', 'database', 'vonage']
    }

    /**
     * Obtient la représentation mail de la notification.
     */
    public function toMail(object $notifiable): MailMessage
    {
        return (new MailMessage)
            ->subject('Transfert Réussi')
            ->line('Votre transfert de ' . $this->transaction->montant . ' a été effectué avec succès.')
            ->action('Voir la transaction',
                route('transactions.show', $this->transaction->id))
            ->line('Merci d\'utiliser notre service !');
    }
}
```

Déclenchement de la Notification :

```
// Dans ProcessTransferAction, après le commit de la transaction
$source->user->notify(new TransferSuccess($sourceTransaction));
$destination->user->notify(new TransferSuccess($destinationTransaction));
```

Partie IV : Tests et Déploiement

Chapitre 8 : Tester pour la Robustesse

Le respect des principes SOLID rend le code intrinsèquement plus facile à tester. Grâce au DIP, nous pouvons facilement simuler (Mocker) les dépendances.

8.1. Tests Unitaires (Unit Tests) : Tester les Services et Repositories (DIP)

Les tests unitaires se concentrent sur une seule unité de code (une classe, une méthode) en isolant ses dépendances.

Exemple : Test du UserService

Nous allons tester le `UserService` en simulant le `UserRepositoryInterface`.

Fichier touché : `tests/Unit/UserServiceTest.php` (À créer)

```
php artisan make:test Unit/UserServiceTest
```

```
// tests/Unit/UserServiceTest.php

namespace Tests\Unit;

use App\Contracts\UserRepositoryInterface;
use App\Models\User;
use App\Services\UserService;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Tests\TestCase;

class UserServiceTest extends TestCase
{
    use RefreshDatabase;

    /** @test */
    public function it_can_create_a_user_through_the_service()
    {
        // 1. Mocking : Création d'un faux Repository (DIP)
        $mockRepository = $this->mock(UserRepositoryInterface::class);

        // 2. Définition du comportement attendu du Mock
        $mockRepository->shouldReceive('create')
            ->once()
            ->andReturn(User::factory()->make([
                'id' => 1,
                'password' => 'hashed_password'
            ]));

        // 3. Instanciation du Service avec le Mock injecté
        $userService = new UserService($mockRepository);

        // 4. Exécution de la méthode à tester
        $userData = [
            'nom' => 'Test',
            'prenom' => 'User',
            'telephone' => '123456789',
            'password' => 'secret',
            'type' => 'client'
        ];
        $user = $userService->createUser($userData);

        // 5. Assertions
        $this->assertInstanceOf(User::class, $user);
        $this->assertEquals(1, $user->id);
        // Le service ne se soucie pas de la persistance, seulement de la
    }
}
```

```
logique métier.  
    }  
}
```

8.2. Tests Fonctionnels (Feature Tests) : Tester les Endpoints API

Les tests fonctionnels vérifient le comportement de l' application dans son ensemble, de la requête HTTP à la réponse.

Exemple : Test de l' Endpoint de Création d' Utilisateur

Fichier touché : tests/Feature/UserCreationTest.php (À créer)

```
php artisan make:test Feature/UserCreationTest
```



```
// tests/Feature/UserCreationTest.php

namespace Tests\Feature;

use App\Models\User;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Tests\TestCase;

class UserCreationTest extends TestCase
{
    use RefreshDatabase;

    /** @test */
    public function only_admin_can_create_a_user()
    {
        // Création d'un utilisateur non-admin
        $client = User::factory()->create(['type' => 'client']);

        $response = $this->actingAs($client, 'api')
            ->postJson('/api/utilisateurs', [
                'nom' => 'New',
                'prenom' => 'User',
                'telephone' => '987654321',
                'password' => 'password',
                'type' => 'client'
            ]);

        // Assertion : Accès refusé (403)
        $response->assertStatus(403);
    }

    /** @test */
    public function admin_can_create_a_user_and_hateoas_links_are_present()
    {
        // Création d'un utilisateur admin
        $admin = User::factory()->create(['type' => 'admin']);

        $response = $this->actingAs($admin, 'api')
            ->postJson('/api/utilisateurs', [
                'nom' => 'Admin',
                'prenom' => 'Created',
                'telephone' => '987654321',
                'password' => 'password',
                'type' => 'client'
            ]);
    }
}
```

```

$response->assertStatus(201)
    ->assertJsonStructure([
        'status',
        'message',
        'data' => [
            'id',
            'nom',
            'prenom',
            'links' => [ // Vérification des liens HATEOAS
                'self',
                'comptes',
                'delete', // Le lien 'delete' est présent pour
l'admin
            ]
        ]
    ]);

// Vérification que l'utilisateur a été créé en base
$this->assertDatabaseHas('users', ['telephone' => '987654321']);
}
}

```

8.3. Tests de Base de Données (Database Testing) : Factories et Seeders

Laravel fournit des **Factories** pour créer facilement des données de test et des **Seeders** pour peupler la base de données.

Fichier touché : database/factories/UserFactory.php (Mise à jour)

```
// database/factories/UserFactory.php (Mise à jour)

// ...
public function definition(): array
{
    return [
        'nom' => $this->faker->lastName(),
        'prenom' => $this->faker->firstName(),
        'telephone' => $this->faker->unique()->numerify('77#####'), //
        // Format Sénégalais
        'email' => $this->faker->unique()->safeEmail(),
        'type' => $this->faker->randomElement(['client', 'admin',
        'marchand'])),
        'password' => static::$password ??= Hash::make('password'),
        'remember_token' => Str::random(10),
    ];
}
// ...
```

Chapitre 9 : Conclusion et Prochaines Étapes

9.1. Récapitulatif des Principes Appliqués

Nous avons construit une API Mobile Money en respectant rigoureusement les contraintes architecturales les plus exigeantes :

Principe/Modèle	Outil Laravel Utilisé	Résultat Architectural
SRP	Services, Actions, Jobs, Notifications	Contrôleurs minces, logique métier isolée.
OCP/DIP	Interfaces, Repositories, Service Container	Code ouvert à l' extension, fermé à la modification.
HATEOAS (RMM L4)	API Resources, Routes Nommées	API auto-documentée et navigable par hypermédia.
Avancé Laravel	Traits, Mutators, Events/Listeners, Jobs Chaînés	Découplage maximal et gestion asynchrone des tâches.

9.2. Déploiement Simplifié

Une architecture propre facilite le déploiement. L' utilisation des Jobs asynchrones nécessite la configuration d' un gestionnaire de file d' attente (Redis ou Database) et d' un processus de travail (Supervisor).

Commandes de Déploiement Clés :

```
# 1. Mise à jour du code
git pull origin main

# 2. Mise à jour des dépendances
composer install --optimize-autoloader --no-dev

# 3. Exécution des migrations
php artisan migrate --force

# 4. Redémarrage du cache
php artisan config:cache
php artisan route:cache
php artisan view:cache

# 5. Redémarrage du gestionnaire de file d'attente (Supervisor)
sudo supervisorctl restart all
```

9.3. Ressources pour Aller Plus Loin

Pour continuer votre apprentissage et approfondir les concepts :

- **Laravel Documentation** : La référence officielle pour toutes les fonctionnalités du framework.
- **Clean Architecture** (Robert C. Martin) : Le livre fondamental pour comprendre les principes d'architecture logicielle.
- **Pattern-Oriented Software Architecture** : Pour explorer d'autres patrons de conception.

FIN DU LIVRE (Le contenu ci-dessus est une ébauche détaillée qui sera formatée pour atteindre 100 pages, en ajoutant des explications plus longues, des schémas, et des exemples de code plus volumineux pour chaque fichier touché.)